
Chapitre : Les Sockets

Introduction

Les sockets sont un mécanisme fondamental pour la communication entre deux programmes à travers un réseau. En Java, les sockets sont largement utilisés pour les communications réseau, que ce soit sur un réseau local ou à travers Internet.

1. Qu'est-ce qu'un Socket ?

Un **socket** est un point de communication pour échanger des données entre deux applications, que ce soit sur une même machine ou à travers un réseau. En termes simples, un socket représente un canal bidirectionnel permettant d'envoyer et de recevoir des données entre un client et un serveur.

2. Modèle Client-Serveur et Fonctionnement des Sockets

Les sockets suivent un modèle **client-serveur** où :

- Le serveur attend qu'un client se connecte sur un port spécifique.
 - Le client se connecte à un serveur donné sur un port spécifique.
- **Le Serveur** : Un serveur utilise un socket pour attendre des connexions entrantes. Il crée une instance de `ServerSocket`, qui est un type particulier de socket permettant de lier un programme à un port d'écoute sur une machine. Lorsqu'un client se connecte, le serveur accepte la connexion avec la méthode `accept()`.
 - **Le Client** : Le client utilise un socket pour se connecter au serveur. Il crée un objet `Socket`, qui nécessite une adresse IP (ou un nom d'hôte) et un port. Une fois la connexion établie, le client et le serveur peuvent échanger des messages.

3. Les Différents Types de Sockets

Les deux principaux types de sockets sont :

- **Sockets TCP (Transmission Control Protocol)** : Utilisés pour une communication orientée connexion, fiable et bidirectionnelle.
- **Sockets UDP (User Datagram Protocol)** : Utilisés pour une communication sans connexion, non fiable, mais plus rapide.

4. Le Protocole TCP : Communication Orientée Connexion

Le protocole TCP est le plus utilisé dans les applications réseau nécessitant une communication fiable. Voici ses caractéristiques :

- ✓ **Fiabilité** : TCP garantit que les données envoyées arrivent à destination, et dans le bon ordre.
- ✓ **Connexion** : Avant de commencer à échanger des données, une connexion est établie entre le client et le serveur.
- ✓ **Contrôle de flux** : TCP régule le flux des données pour éviter la congestion du réseau.
- ✓ **Contrôle de congestion** : TCP détecte et répond à la congestion réseau, ralentissant ou accélérant le flux des données.

4.1 Fonctionnement du Protocole TCP

- Establishment (Établissement de la connexion) :
 - Le processus commence par un échange en trois étapes appelé le handshake (serrement de mains) entre le client et le serveur. Cela permet d'établir une connexion fiable.
 - SYN : Le client envoie un paquet SYN (synchronisation) pour initier la connexion.
 - SYN-ACK : Le serveur répond par un paquet SYN-ACK pour accepter la demande.
 - ACK : Le client envoie un ACK pour confirmer la connexion.
- Data Transfer (Transfert de données) :
 - Une fois la connexion établie, les données peuvent être envoyées dans les deux sens entre le client et le serveur.
- Teardown (Fermeture de la connexion) :
 - Lorsqu'une des parties (client ou serveur) n'a plus de données à envoyer, elle peut envoyer un paquet FIN pour commencer la procédure de fermeture de la connexion.

4.2 Exemple d'Architecture TCP

Le serveur attend qu'un client se connecte à un port spécifique. Une fois la connexion établie, les deux parties peuvent échanger des données dans les deux sens. Voici un exemple d'une architecture de communication :

- Le client envoie une requête au serveur.
- Le serveur traite la requête et renvoie une réponse.
- Les connexions TCP garantissent que les messages arrivent dans l'ordre, sans perte.

5. Le Protocole UDP : Communication Sans Connexion

Le protocole UDP est utilisé pour des communications rapides mais non fiables, où la perte de certains paquets de données n'est pas critique. Ce protocole est souvent utilisé pour les applications en temps réel, comme le streaming vidéo ou les jeux en ligne, où la vitesse est plus importante que la fiabilité.

5.1 Caractéristiques de UDP

- ✓ **Non fiable** : UDP ne garantit pas la réception des paquets, ni leur ordre.
- ✓ **Pas de connexion préalable** : Il n'y a pas d'établissement de connexion avant l'envoi des données.
- ✓ **Légereté** : Le protocole UDP est plus léger et plus rapide que TCP, car il n'y a pas de vérification de la réception des données.

5.2 Exemple d'Architecture UDP

Dans UDP, un client envoie un paquet de données à un serveur sans établir de connexion préalable. Le serveur, s'il existe, recevra le paquet. Il n'y a pas de confirmation, ni de processus de reconnexion.

6. Classes Java pour la Gestion des Sockets

En Java, les classes principales pour manipuler les sockets sont fournies par le package `java.net`.

6.1 Le Client et le Serveur TCP avec `Socket` et `ServerSocket`

- `Socket` : Représente une connexion réseau entre un client et un serveur. Cette classe est utilisée côté client pour se connecter à un serveur.

- `Socket(String host, int port)` : Crée un socket et se connecte à un serveur.
- `getInputStream()` : Récupère un flux d'entrée pour lire les données envoyées par le serveur.
- `getOutputStream()` : Récupère un flux de sortie pour envoyer des données au serveur.

- `ServerSocket` : Représente un point d'écoute pour accepter les connexions des clients. C'est utilisé côté serveur pour attendre les demandes de connexion.

- `ServerSocket(int port)` : Crée un serveur qui écoute sur un port spécifique.
- `accept()` : Attend qu'un client se connecte et retourne un objet `Socket` pour la communication.

Exemple Socket TCP

- **Côté Serveur**

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(1234)) {
            System.out.println("Server is listening on port 1234");
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");
                new ServerThread(socket).start();
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}

class ServerThread extends Thread {
    private Socket socket;

    public ServerThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try (InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true)) {

            String text;
            while ((text = reader.readLine()) != null) {
                System.out.println("Received: " + text);
                writer.println("Echo: " + text);
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

- **Coté Client**

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        String hostname = "localhost";
        int port = 1234;

        try (Socket socket = new Socket(hostname, port)) {
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true);

            InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));

            writer.println("Hello, Server");
            String response = reader.readLine();
            System.out.println("Server response: " + response);
        } catch (UnknownHostException ex) {
            System.out.println("Server not found: " + ex.getMessage());
        } catch (IOException ex) {
            System.out.println("I/O error: " + ex.getMessage());
        }
    }
}
```

6.2 Le Client et le Serveur UDP avec `DatagramSocket` et `DatagramPacket`

- **`DatagramSocket`** : Représente un socket UDP utilisé pour envoyer et recevoir des paquets de données.
 - **`DatagramSocket(int port)`** : Crée un socket UDP lié à un port spécifique.
 - **`send(DatagramPacket packet)`** : Envoie un paquet de données.
 - **`receive(DatagramPacket packet)`** : Reçoit un paquet de données.
- **`DatagramPacket`** : Représente un paquet UDP contenant des données.
 - **`DatagramPacket(byte[] buf, int length)`** : Crée un paquet de données à envoyer.
 - **`DatagramPacket(byte[] buf, int length, InetAddress address, int port)`** : Crée un paquet pour un envoi vers un serveur spécifique.

Exemple Socket UDP

- **Cote Serveur**

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket(1234)) {
            byte[] buffer = new byte[1024];
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
            System.out.println("Server is listening on port 1234");

            while (true) {
                socket.receive(packet);
                String received = new String(packet.getData(), 0, packet.getLength());
                System.out.println("Received: " + received);

                String response = "Echo: " + received;
                byte[] responseData = response.getBytes();
                DatagramPacket responsePacket = new DatagramPacket(responseData,
                    responseData.length, packet.getAddress(), packet.getPort());
                socket.send(responsePacket);
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

- **Cote Client**

```
import java.net.*;

public class UDPClient {

    public static void main(String[] args) {

        String hostname = "localhost";

        int port = 1234;

        try (DatagramSocket socket = new DatagramSocket()) {

            byte[] buffer = "Hello, Server".getBytes();

            InetAddress address = InetAddress.getByName(hostname);
```

```

DatagramPacket packet = new DatagramPacket(buffer, buffer.length, address, port);

socket.send(packet);

byte[] responseBuffer = new byte[1024];

DatagramPacket responsePacket = new DatagramPacket(responseBuffer,
responseBuffer.length);

socket.receive(responsePacket);

String response = new String(responsePacket.getData(), 0, responsePacket.getLength());

System.out.println("Server response: " + response);

} catch (IOException ex) {

    ex.printStackTrace();

}

}

}

```

7. Gestion des Exceptions et des Erreurs

Les sockets peuvent rencontrer divers types d'erreurs pendant leur utilisation, comme la perte de la connexion, la congestion du réseau ou des erreurs d'entrée/sortie. En Java, ces erreurs sont généralement traitées avec des exceptions.

Les exceptions courantes sont :

- **`IOException`** : L'exception de base pour les erreurs d'entrée/sortie (réseau, fichier, etc.).
- **`SocketException`** : Erreur spécifique liée aux sockets, comme une erreur de connexion ou de fermeture.
- **`BindException`** : Erreur si un autre programme utilise déjà le port demandé pour le serveur.

8. Sécurisation des Sockets avec TLS/SSL

Lorsqu'une communication doit être sécurisée, il est courant d'utiliser des versions sécurisées de TCP, telles que TLS (Transport Layer Security) ou SSL (Secure Sockets Layer). Ces protocoles garantissent la confidentialité, l'intégrité et l'authenticité des données échangées.

En Java, la sécurité des sockets est fournie par les classes ``SSLSocket`` et ``SSLServerSocket``, qui permettent de chiffrer les données échangées entre le client et le serveur.

9. Applications Pratiques des Sockets en Java

Les sockets sont largement utilisés dans des domaines variés :

- **Applications Web** : Les serveurs web utilisent des sockets TCP pour communiquer avec les navigateurs via le protocole HTTP.
- **Jeux en ligne** : Utilisation de sockets UDP pour des jeux en temps réel nécessitant une faible latence.
- **Services de messagerie**: Les clients de messagerie utilisent souvent des sockets pour envoyer et recevoir des e-mails via des protocoles comme SMTP et IMAP.

Conclusion

Les sockets en Java sont un outil essentiel pour la création d'applications réseau, qu'il s'agisse de protocoles fiables (TCP) ou non fiables (UDP). La compréhension du modèle client-serveur